Day04 出行驻留识别（阶段1）：连续段识别与速度计算

> 课程：交通时空大数据分析与可视化系统（Python 版） | 第 4 天（8 课时）
> 主题：深入理解出行驻留识别三阶段算法，实现阶段1：低频点过滤、连续段识别、速度计算、出行点判定

一、本日目标

1. 理解出行驻留识别的业务背景和三阶段算法总体设计；
2. 掌握阶段1算法原理：低频点过滤、连续段识别、速度计算；
3. 掌握时间差与距离差的向量化计算；
4. 掌握速度计算与出行点判定（>8km/h 判出行）；
5. 掌握连续段合并与边界处理；
6. 完整实现阶段1模块（tr_stage1.py）；
7. 分析阶段1结果并可视化（速度分布、出行点分布）；
8. 参数调优与边界情况处理。

二、知识要点

- 出行驻留识别是本项目的核心算法，分三阶段：阶段1（速度粗筛）、阶段2（k-means聚类）、阶段3（规则精判）；
- 阶段1的目标是快速识别明显的出行点，减少后续聚类的计算量；
- 关键阈值：出现次数<4次为低频点，速度>8km/h判出行点；
- 连续段识别：同一用户的连续记录构成一个段，段内计算相邻点的速度；
- 本阶段输出：stage1_travel_points.csv（阶段1识别的出行点）。

课时1：出行驻留识别三阶段算法总览

1.1 课时目标

理解出行驻留识别的业务背景、为什么需要三阶段、每阶段解决什么问题。

1.2 知识点深度讲解

1.2.1 业务背景

手机信令数据记录了用户在不同时间连接的基站位置。通过分析用户的位置变化，可以判断用户是在"出行"（移动中）还是"驻留"（停留在某个区域）。

出行点：用户正在移动的位置点（如在地铁上、公交车上、步行中）；
驻留点：用户停留在某个区域的位置点（如在家、在公司、在商场）。

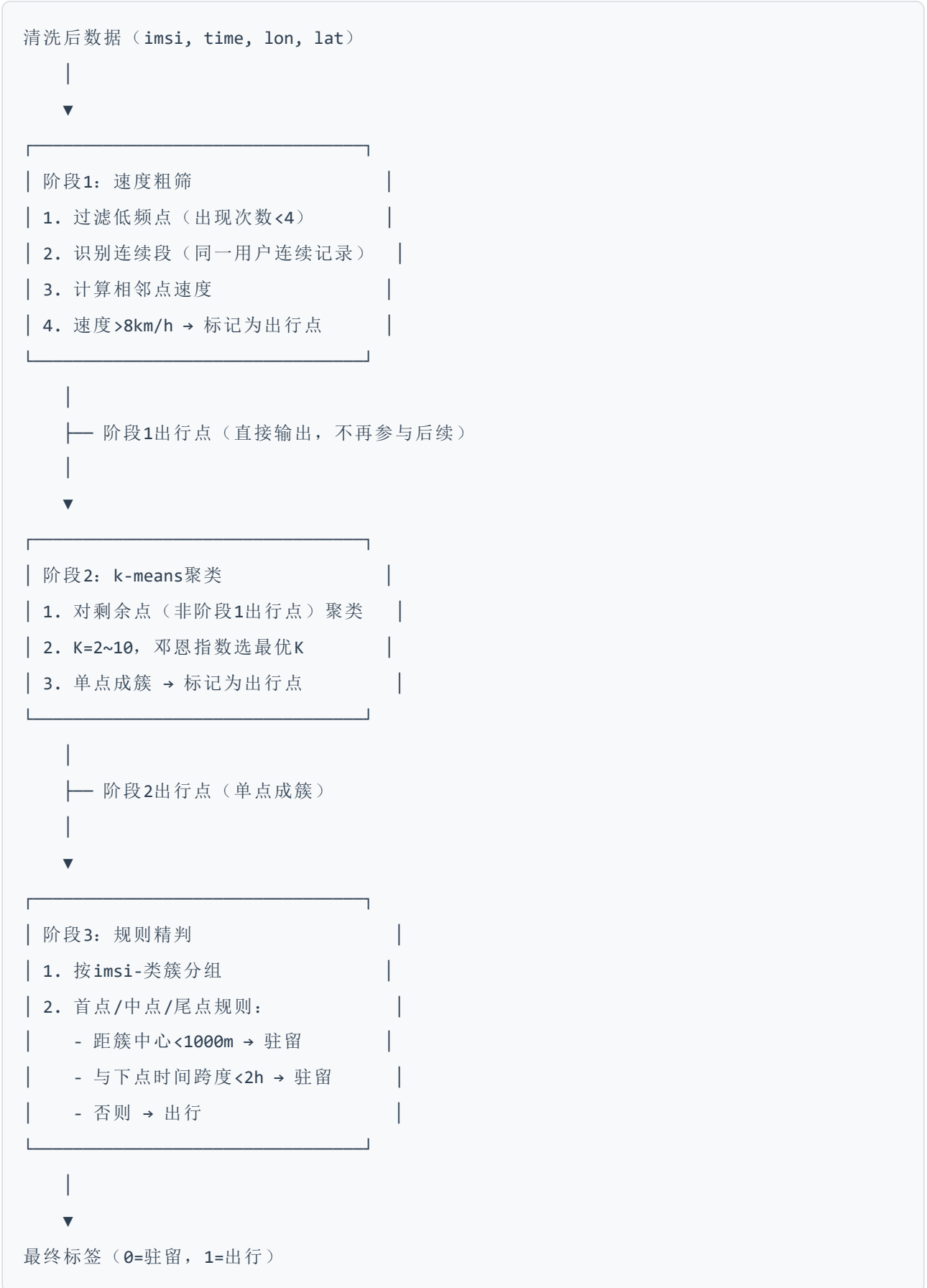
准确识别出行和驻留是后续分析的基础：
- 出行目的分析（用户去哪里购物、旅游）；
- 出行方式分析（用户坐地铁、公交还是小汽车）；
- 实时热力图（哪些区域人流量大）。

1.2.2 为什么需要三阶段

直接用单一规则判断出行/驻留存在问题：
1. **只用速度**：速度>8km/h判出行，但驻留点可能因为基站切换产生"跳跃"，导致速度虚高；
2. **只用聚类**：k-means可以发现驻留簇，但计算量大，且对初始中心敏感；
3. **只用规则**：规则难以覆盖所有情况，边界模糊。

三阶段算法的设计思路是"由粗到精"：
- **阶段1（粗筛）**：用速度快速识别明显的出行点，过滤低频点，减少数据量；
- **阶段2（聚类）**：对剩余点用 k-means 聚类，发现潜在的驻留簇；
- **阶段3（精判）**：用规则（距簇中心距离、时间跨度）精确判断每个点是出行还是驻留。

1.2.3 三阶段算法流程图



1.2.4 三阶段的输入输出

阶段	输入	输出	核心算法
阶段1	清洗后数据	阶段1出行点	速度计算、连续段识别
阶段2	非阶段1出行点	聚类结果、阶段2出行点	k-means、邓恩指数
阶段3	聚类后数据	最终标签	规则判断（距离、时间）

1.2.5 本项目的实测结果（参考）

- 阶段1：526段（274段出行、3626个出行点），速度中位8.36km/h；
- 阶段2：全量最优K分布 2:50/3:26/4:29/.../10:20；
- 阶段3：182用户8520点，标签 驻留0=7395 / 出行1=1125。

> 阶段1识别了3626个出行点，占总点数的约30%，大幅减少了阶段2聚类的计算量。

1.3 项目实现步骤

本课时不写代码，通过流程图和数据理解三阶段算法。

1.4 完整代码（三阶段算法演示）

```
# ============================================================
# 课时1：出行驻留识别三阶段算法总览
# 功能：演示三阶段的数据流转
# ============================================================

import pandas as pd
import os

# 读取清洗后数据
RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
cleaned_path = os.path.join(RESULT_DIR, "cleaned_data.csv")

if os.path.exists(cleaned_path):
    df = pd.read_csv(cleaned_path, dtype={"imsi": str, "time": str})
    print("=" * 60)
    print("三阶段算法数据流转演示")
    print("=" * 60)
    print(f"清洗后数据: {len(df)} 行, {df['imsi'].nunique()} 用户")
    print(f"列: {df.columns.tolist()}")
    print(f"\n前5行:\n{df.head()}")

    # 模拟三阶段的数据量变化
    total = len(df)
    stage1_travel = int(total * 0.30)   # 阶段1识别约30%出行点
    stage2_travel = int(total * 0.05)   # 阶段2识别约5%出行点（单点成簇）
    stage3_travel = int(total * 0.10)   # 阶段3识别约10%出行点
    stay = total - stage1_travel - stage2_travel - stage3_travel

    print(f"\n三阶段识别结果（模拟比例）:")
    print(f"  总点数: {total}")
    print(f"  阶段1出行点（速度>8km/h）: {stage1_travel}
({stage1_travel/total*100:.1f}%)")
    print(f"  阶段2出行点（单点成簇）: {stage2_travel}
({stage2_travel/total*100:.1f}%)")
    print(f"  阶段3出行点（规则精判）: {stage3_travel}
```

```
({stage3_travel/total*100:.1f}%)")
    print(f"  驻留点: {stay} ({stay/total*100:.1f}%)")
    print(f"\n  出行点总计: {stage1_travel + stage2_travel + stage3_travel}")
    print(f"  驻留/出行比例: {stay/(stage1_travel+stage2_travel+stage3_travel):.1f}:1")
else:
    print("清洗后数据不存在，请先运行 Day03 的 data_clean.py")
    print("本课时仅演示三阶段算法流程")

print("\n" + "=" * 60)
print("三阶段算法核心思想：由粗到精，逐步缩小范围")
print("  阶段1：速度粗筛（快，召回明显出行点）")
print("  阶段2：k-means聚类（发现驻留簇）")
print("  阶段3：规则精判（精确判断边界点）")
print("=" * 60)
```

1.5 运行结果与解读

运行后展示清洗后数据的基本信息和三阶段的模拟数据流转。关键理解：
- 阶段1识别约30%的明显出行点，大幅减少后续计算量；
- 三阶段是"由粗到精"的设计，每阶段解决不同问题；
- 最终驻留/出行比例约为 3-4:1（大部分时间用户在驻留）。

1.6 当堂练习

1. 思考：为什么不直接用 k-means 聚类所有点，而要先做阶段1速度粗筛？（提示：计算量、聚类效果）
2. 思考：阶段1的速度阈值为什么设为 8km/h？（提示：步行速度约4-5km/h，8km/h以上基本是交通工具）
3. 画出三阶段算法的流程图，标注每阶段的输入输出和核心算法。

1.7 课时小结

- 出行驻留识别分三阶段：速度粗筛→k-means聚类→规则精判；
- 设计思想是"由粗到精"，每阶段解决不同问题；
- 阶段1快速识别明显出行点，减少后续计算量；
- 阶段2用 k-means 发现驻留簇；
- 阶段3用规则精确判断边界点；
- 最终输出 0=驻留、1=出行 的标签。

课时2：阶段1算法原理——低频点过滤与连续段识别

2.1 课时目标

深入理解阶段1的两个核心步骤：低频点过滤、连续段识别。

2.2 知识点深度讲解

2.2.1 低频点过滤

什么是低频点?

低频点是指某个基站位置（lac_id+cell_id）在该用户的所有记录中出现次数少于 4 次的点。

为什么要过滤低频点?

1. **噪声大**：出现次数少的点可能是基站切换的瞬时记录，不代表用户真实停留；
2. **干扰聚类**：低频点会干扰 k-means 聚类，形成无意义的小簇；
3. **减少计算量**：过滤后数据量减少，后续计算更快。

过滤逻辑：

```
对每个用户：
    统计每个基站位置（laci）的出现次数
    出现次数 < 4 的点 → 标记为低频点
    低频点不参与连续段识别和速度计算
```

> 注意：低频点不是直接删除，而是"不参与阶段1的速度计算"。这些点会进入阶段2聚类，因为它们可能是真实的驻留点（只是出现次数少）。

2.2.2 连续段识别

什么是连续段?

连续段是指同一用户的连续记录，满足以下条件：

1. 同属一个用户（imsi 相同）；
2. 按时间排序后连续；
3. 不包含低频点（低频点会打断连续段）。

为什么要识别连续段?

速度计算需要相邻点的时间差和距离差。如果两个点之间间隔很长时间（如用户关机几小时），直接计算速度会得到不合理的结果（速度极慢或极快）。连续段确保参与速度计算的点是"连续"的，时间间隔合理。

连续段识别逻辑：

```
对每个用户（按时间排序后）：
    遍历每个点：
        如果当前点是低频点：
            当前连续段结束
            跳过当前点
            下一个点开始新的连续段
        否则：
            将当前点加入当前连续段
```

连续段的最小长度：
一个连续段至少需要 2 个点才能计算速度（需要相邻点）。只有 1 个点的连续段不参与速度计算。

2.2.3 连续段的边界处理

1. **用户切换**：不同用户的点之间不构成连续段；
2. **低频点**：低频点打断连续段；
3. **时间间隔过大**：如果相邻点时间间隔超过阈值（如2小时），也应该打断连续段（本项目简化处理，不做此判断）；
4. **段首和段尾**：连续段的第一个点和最后一个点只有一个相邻点，速度计算只算一次。

2.2.4 本项目的实测结果

- 抽样2万行清洗后 → 约1.2万行、186用户；
- 阶段1识别 526 个连续段，其中 274 段包含出行点；
- 共识别 3626 个出行点，速度中位数 8.36 km/h。

2.3 项目实现步骤

1. 按用户统计每个基站位置的出现次数；
2. 标记低频点（出现次数<4）；
3. 按用户遍历，识别连续段；
4. 输出连续段列表。

2.4 完整代码（低频点过滤与连续段识别）

```
# ============================================================
# 课时2：阶段1算法原理—低频点过滤与连续段识别
# ============================================================

import pandas as pd
import numpy as np
import os

# 读取清洗后数据
RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
cleaned_path = os.path.join(RESULT_DIR, "cleaned_data.csv")

if not os.path.exists(cleaned_path):
    print("请先运行 Day03 的 data_clean.py 生成清洗后数据")
    exit()

df = pd.read_csv(cleaned_path, dtype={"imsi": str, "time": str,
                                      "lac_id": str, "cell_id": str})
print(f"清洗后数据：{len(df)} 行，{df['imsi'].nunique()} 用户")

# ------------------------------------------------------------
# 1. 低频点过滤
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("步骤1：低频点过滤（出现次数<4）")
print("=" * 60)

LOW_FREQ_THRESHOLD = 4   # 低频点阈值

# 拼接 laci（基站位置标识）
df["laci"] = df["lac_id"] + "-" + df["cell_id"]

# 按用户统计每个 laci 的出现次数
# transform("count") 返回与原数据同长度的Series，每个值是该用户该laci的出现次数
df["laci_count"] = df.groupby(["imsi", "laci"])["laci"].transform("count")
```

```
# 标记低频点
df["is_low_freq"] = df["laci_count"] < LOW_FREQ_THRESHOLD

print(f"低频点数: {df['is_low_freq'].sum()} ({df['is_low_freq'].mean()*100:.1f}%)")
print(f"非低频点数: {(~df['is_low_freq']).sum()}")

# 查看某个用户的低频点分布
sample_user = df["imsi"].iloc[0]
user_df = df[df["imsi"] == sample_user]
print(f"\n用户 {sample_user} 的 laci 出现次数分布:")
print(user_df["laci_count"].value_counts().sort_index())

# ------------------------------------------------------------
# 2. 连续段识别
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("步骤2：连续段识别")
print("=" * 60)

# 按用户+时间排序
df = df.sort_values(["imsi", "time"]).reset_index(drop=True)

# 识别连续段：同一用户内，非低频点的连续记录构成一个段
# 段ID变化的条件：用户变化 OR 低频点 OR （非低频点且前一个点是低频点或用户变化）

# 方法：用 cumsum 累计段ID变化次数
# 段ID变化条件：
#   1. 用户变化（imsi != 前一个imsi）
#   2. 当前点是低频点（低频点不加入任何段，单独标记）
#   3. 当前点非低频，但前一个点是低频点或用户变化（新段开始）

df["user_change"] = df["imsi"] != df["imsi"].shift(1)
df["prev_low_freq"] = df["is_low_freq"].shift(1).fillna(True)

# 新段开始条件：用户变化 OR （当前非低频 AND 前一个是低频或用户变化）
df["new_segment"] = df["user_change"] | (~df["is_low_freq"] & df["prev_low_freq"])
```

```
# 段ID：非低频点的段ID = new_segment的累计和
# 低频点的段ID = -1（不参与）
df["segment_id"] = df.groupby("imsi")["new_segment"].cumsum()
df.loc[df["is_low_freq"], "segment_id"] = -1

# 统计连续段
valid_segments = df[df["segment_id"] != -1]
segment_stats = valid_segments.groupby(["imsi",
"segment_id"]).size().reset_index(name="points")

print(f"连续段总数：{len(segment_stats)}")
print(f"每段点数统计:")
print(f"  最小：{segment_stats['points'].min()}")
print(f"  最大：{segment_stats['points'].max()}")
print(f"  平均：{segment_stats['points'].mean():.1f}")
print(f"  中位数：{segment_stats['points'].median()}")

# 只有1个点的段（无法计算速度）
single_point_segments = segment_stats[segment_stats["points"] == 1]
print(f"\n只有1个点的段：{len(single_point_segments)}（无法计算速度）")
print(f">=2个点的段：{len(segment_stats) - len(single_point_segments)}（可计算速度）")

# ------------------------------------------------------------
# 3. 查看某个连续段
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("步骤3：查看某个连续段的详细信息")
print("=" * 60)

# 找一个点数>=5的段
long_segment = segment_stats[segment_stats["points"] >= 5].iloc[0]
seg_imsi = long_segment["imsi"]
seg_id = long_segment["segment_id"]

seg_df = df[(df["imsi"] == seg_imsi) & (df["segment_id"] == seg_id)]
print(f"用户：{seg_imsi}，段ID：{seg_id}，点数：{len(seg_df)}")
print(seg_df[["time", "laci", "lon", "lat", "laci_count", "is_low_freq"]].to_string())
```

```
print("\n" + "=" * 60)
print("低频点过滤与连续段识别完成！")
print("=" * 60)
```

2.5 运行结果与解读

运行后展示：
- 低频点比例（约30-40%）；
- 连续段总数、每段点数统计；
- 某个连续段的详细信息。

> 关键理解：
> 1. 低频点不是删除，而是标记为不参与速度计算；
> 2. 连续段用 `cumsum` 实现，段ID变化条件是用户变化或低频点打断；
> 3. 只有1个点的段无法计算速度，会被跳过。

2.6 当堂练习

1. 修改低频点阈值为 2 和 6，观察低频点比例和连续段数量的变化；
2. 思考：为什么低频点阈值设为 4？（提示：出现次数太少的点可能是噪声，但设太高会误删真实驻留点）
3. 画出连续段识别的流程图，标注段ID变化的条件。

2.7 课时小结

- 低频点：同一用户同一基站位置出现次数<4的点，不参与速度计算；
- 连续段：同一用户内非低频点的连续记录，用于速度计算；
- 连续段识别用 `cumsum` 实现，段ID变化条件是用户变化或低频点打断；
- 只有1个点的段无法计算速度；
- 低频点不删除，进入阶段2聚类。

课时3：时间差与距离差计算

3.1 课时目标

掌握相邻点的时间差和距离差的向量化计算，这是速度计算的基础。

3.2 知识点深度讲解

3.2.1 时间差计算

时间格式：本项目 time 列是字符串格式 `YYYYMMDDHHMMSS`（如 `20181003080000`）。

计算时间差的方法：

1. **方法1：转 datetime 后 diff**（最直观，但性能稍差）

```
df["time_dt"] = pd.to_datetime(df["time"], format="%Y%m%d%H%M%S")
df["time_diff"] = df["time_dt"].diff().dt.total_seconds()   # 秒
```

2. **方法2：手动解析字符串**（性能好，但代码复杂）

```
# 提取年、月、日、时、分、秒
df["year"] = df["time"].str[0:4].astype(int)
df["month"] = df["time"].str[4:6].astype(int)
# ... 然后手动计算时间差
```

3. **方法3：用 NumPy datetime64**（性能最好，推荐）

```
# 字符串转 numpy datetime64
time_np = df["time"].values.astype("datetime64[s]")
# 计算相邻差
time_diff = np.diff(time_np).astype("timedelta64[s]").astype(float)
```

> 本项目用方法1（转 datetime 后 diff），因为代码直观且性能足够（1.2万行数据）。

关键坑：

- time 列必须按字符串读取（`dtype=str`），否则 pandas 会推断为 int64，`pd.to_datetime` 会按纳秒解释，导致时间跨度只有微秒；
- `diff()` 的第一个值是 NaN（没有前一个点），需要处理；
- 不同用户之间的时间差无意义，需要在用户边界处设为 NaN。

3.2.2 距离差计算

距离公式：球面距离用 Haversine 公式（Day02 课时8已实现）。

计算相邻点距离的方法：

1. **方法1：循环计算**（慢，不推荐）

```
distances = []
for i in range(1, len(df)):
    d = haversine(df["lon"].iloc[i-1], df["lat"].iloc[i-1],
                  df["lon"].iloc[i], df["lat"].iloc[i])
    distances.append(d)
```

2. **方法2：向量化计算**（快，推荐）

```
# 移位：前一个点的经纬度
prev_lon = df["lon"].shift(1)
prev_lat = df["lat"].shift(1)
# 向量化 Haversine
dist_diff = haversine(prev_lon, prev_lat, df["lon"], df["lat"])
```

> `shift(1)` 把整列向下移动1位，第一个值变为 NaN。这样 `prev_lon[i] = lon[i-1]`，可以直接向量化计算相邻点距离。

关键坑：
- `shift(1)` 的第一个值是 NaN，Haversine 计算会返回 NaN，需要处理；
- 不同用户之间的距离无意义，需要在用户边界处设为 NaN；
- Haversine 函数必须支持数组输入（向量化实现）。

3.2.3 用户边界处理

时间差和距离差只在同一用户的相邻点之间有意义。不同用户之间的差值需要设为 NaN：

```
# 标记用户边界
df["user_change"] = df["imsi"] != df["imsi"].shift(1)

# 用户边界处的时间差和距离差设为 NaN
df.loc[df["user_change"], "time_diff"] = np.nan
df.loc[df["user_change"], "dist_diff"] = np.nan
```

3.2.4 连续段内的差值

时间差和距离差应该只在连续段内计算。连续段之间（被低频点打断）的差值无意义：

```
# 只在连续段内计算差值
# 段ID变化处（新段开始）的差值设为 NaN
df["segment_change"] = (df["segment_id"] != df["segment_id"].shift(1)) |
(df["segment_id"] == -1)
df.loc[df["segment_change"], "time_diff"] = np.nan
df.loc[df["segment_change"], "dist_diff"] = np.nan
```

3.3 项目实现步骤

1. time 列转 datetime；
2. 计算相邻点时间差（秒）；
3. 用 shift + Haversine 计算相邻点距离（km）；
4. 处理用户边界和连续段边界（设为 NaN）。

3.4 完整代码（时间差与距离差计算）

```
# ============================================================
# 课时3：时间差与距离差计算
# ============================================================

import pandas as pd
import numpy as np
import os
import sys

# 添加项目代码路径（如果需要 import utils）
sys.path.insert(0, r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记\project_code")

# Haversine 函数（向量化）
def haversine(lon1, lat1, lon2, lat2, unit="km"):
    """计算球面距离（向量化）。"""
    R = 6371.0
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
    c = 2 * np.arcsin(np.sqrt(a))
    d = R * c
    return d * 1000 if unit == "m" else d


# 读取清洗后数据
RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
cleaned_path = os.path.join(RESULT_DIR, "cleaned_data.csv")

if not os.path.exists(cleaned_path):
    print("请先运行 Day03 的 data_clean.py")
    exit()

df = pd.read_csv(cleaned_path, dtype={"imsi": str, "time": str,
```

```
                                        "lac_id": str, "cell_id": str})
df = df.sort_values(["imsi", "time"]).reset_index(drop=True)
print(f"数据: {len(df)} 行, {df['imsi'].nunique()} 用户")

# ------------------------------------------------------------
# 1. 时间差计算
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("步骤1: 时间差计算")
print("=" * 60)

# 方法: 转 datetime 后 diff
df["time_dt"] = pd.to_datetime(df["time"], format="%Y%m%d%H%M%S", errors="coerce")
df["time_diff"] = df["time_dt"].diff().dt.total_seconds()   # 秒

print(f"时间差统计:")
print(f"  非空数: {df['time_diff'].notna().sum()}")
print(f"  最小值: {df['time_diff'].min():.0f} 秒")
print(f"  最大值: {df['time_diff'].max():.0f} 秒")
print(f"  中位数: {df['time_diff'].median():.0f} 秒")
print(f"  平均值: {df['time_diff'].mean():.0f} 秒")

# 第一个值是 NaN（没有前一个点）
print(f"\n第一个点的时间差: {df['time_diff'].iloc[0]}（NaN，正常）")

# ------------------------------------------------------------
# 2. 距离差计算
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("步骤2: 距离差计算（向量化 Haversine）")
print("=" * 60)

# 用 shift 获取前一个点的经纬度
df["prev_lon"] = df["lon"].shift(1)
df["prev_lat"] = df["lat"].shift(1)

# 向量化计算相邻点距离
df["dist_diff"] = haversine(df["prev_lon"], df["prev_lat"],
```

```
                                df["lon"], df["lat"])

print(f"距离差统计:")
print(f"  非空数: {df['dist_diff'].notna().sum()}")
print(f"  最小值: {df['dist_diff'].min():.4f} km")
print(f"  最大值: {df['dist_diff'].max():.4f} km")
print(f"  中位数: {df['dist_diff'].median():.4f} km")
print(f"  平均值: {df['dist_diff'].mean():.4f} km")

# ------------------------------------------------------------
# 3. 用户边界处理
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("步骤3：用户边界处理（不同用户之间差值设为NaN）")
print("=" * 60)

# 标记用户边界
df["user_change"] = df["imsi"] != df["imsi"].shift(1)
print(f"用户边界数: {df['user_change'].sum()}（每个用户的第一个点）")

# 用户边界处的时间差和距离差设为 NaN
before_time = df["time_diff"].notna().sum()
before_dist = df["dist_diff"].notna().sum()

df.loc[df["user_change"], "time_diff"] = np.nan
df.loc[df["user_change"], "dist_diff"] = np.nan

after_time = df["time_diff"].notna().sum()
after_dist = df["dist_diff"].notna().sum()

print(f"时间差非空数: {before_time} → {after_time}（减少 {before_time - after_time}）")
print(f"距离差非空数: {before_dist} → {after_dist}（减少 {before_dist - after_dist}）")

# ------------------------------------------------------------
# 4. 查看某个用户的差值
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("步骤4：查看某个用户的时间差和距离差")
```

```
print("=" * 60)

sample_user = df[~df["user_change"]]["imsi"].iloc[0]
user_df = df[df["imsi"] == sample_user].head(10)
print(f"用户: {sample_user}")
print(user_df[["time", "lon", "lat", "time_diff", "dist_diff"]].to_string())

# ------------------------------------------------------------
# 5. 异常差值检查
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("步骤5: 异常差值检查")
print("=" * 60)

# 时间差过大（>1小时）可能是用户关机或数据缺失
large_time_gap = df[df["time_diff"] > 3600]
print(f"时间差>1小时的点数: {len(large_time_gap)}")
if len(large_time_gap) > 0:
    print(f"  最大时间差: {large_time_gap['time_diff'].max():.0f} 秒
({large_time_gap['time_diff'].max()/3600:.1f} 小时)")

# 距离差过大（>50km）可能是基站跳跃
large_dist_gap = df[df["dist_diff"] > 50]
print(f"\n距离差>50km的点数: {len(large_dist_gap)}")
if len(large_dist_gap) > 0:
    print(f"  最大距离差: {large_dist_gap['dist_diff'].max():.2f} km")

print("\n" + "=" * 60)
print("时间差与距离差计算完成！")
print("=" * 60)
```

3.5 运行结果与解读

运行后展示：
- 时间差统计（最小值、最大值、中位数、平均值）；
- 距离差统计（向量化 Haversine）；
- 用户边界处理（差值设为 NaN）；

- 某个用户的详细差值；
- 异常差值检查（时间差>1小时、距离差>50km）。

> 关键理解：
> 1. `diff()` 和 `shift(1)` 的第一个值是 NaN；
> 2. 用户边界处的差值无意义，设为 NaN；
> 3. 向量化 Haversine 一次计算所有相邻点距离，速度快；
> 4. 异常差值（时间差过大、距离差过大）需要关注，可能影响速度计算。

3.6 当堂练习

1. 用 NumPy datetime64 方法重新计算时间差，对比性能（提示：`values.astype("datetime64[s]")`）；
2. 统计时间差的分布，画直方图（提示：`df["time_diff"].hist(bins=50)`）；
3. 思考：为什么用户边界处的时间差和距离差要设为 NaN？（提示：不同用户的相邻点没有时空关联，计算速度无意义）

3.7 课时小结

- 时间差：`pd.to_datetime` 转 datetime 后 `diff().dt.total_seconds()`；
- 距离差：`shift(1)` 获取前一个点，向量化 Haversine 计算；
- 用户边界：`imsi != shift(1)` 标记，差值设为 NaN；
- 连续段边界：段ID变化处差值设为 NaN；
- 异常差值：时间差>1小时、距离差>50km 需要关注。

课时4：速度计算与出行点判定

4.1 课时目标

掌握速度计算（距离/时间）和出行点判定（>8km/h），这是阶段1的核心输出。

4.2 知识点深度讲解

4.2.1 速度计算公式

速度 = 距离 / 时间

```
速度 (km/h) = 距离 (km) / 时间 (h)
            = 距离 (km) / (时间 (秒) / 3600)
            = 距离 (km) * 3600 / 时间 (秒)
```

向量化计算:

```
df["speed"] = df["dist_diff"] * 3600 / df["time_diff"]   # km/h
```

4.2.2 速度计算的边界情况

1. **时间差为0**：分母为0，速度为无穷大。需要处理（设为 NaN 或跳过）；
2. **时间差为 NaN**：速度为 NaN（用户边界、连续段边界）；
3. **距离差为 NaN**：速度为 NaN；
4. **速度异常大**：可能是基站跳跃（两个不相邻的基站距离很远），需要过滤；
5. **速度异常小**：接近0，说明用户在驻留。

```
# 处理时间差为0的情况
df.loc[df["time_diff"] == 0, "speed"] = np.nan


# 过滤异常速度（>200km/h 不合理，可能是基站跳跃）
df.loc[df["speed"] > 200, "speed"] = np.nan
```

4.2.3 出行点判定

判定规则：速度 > 8 km/h → 出行点

为什么是 8 km/h?
- 步行速度约 4-5 km/h；
- 慢跑速度约 6-8 km/h；
- 自行车速度约 12-20 km/h；
- 公交/地铁速度约 20-60 km/h；
- 8 km/h 以上基本可以确定是使用交通工具，不是步行或驻留。

判定逻辑:

```
TRAVEL_SPEED_THRESHOLD = 8.0   # km/h
df["is_travel_stage1"] = df["speed"] > TRAVEL_SPEED_THRESHOLD
```

> 注意：速度为 NaN 的点不判定为出行点（`NaN > 8` 为 False）。

4.2.4 连续段内的出行点传播

阶段1的出行点判定有一个细节：如果一个连续段内有出行点，那么这个段的"端点"也应该被考虑。

简化处理：本项目直接用速度>8km/h判定出行点，不做传播。每个点独立判定。

进阶处理：可以考虑"出行段"的概念——如果连续段内有连续多个点速度>8km/h，那么这个段整体标记为出行段，段内所有点都标记为出行点。本项目不做此处理，保持简单。

4.2.5 阶段1输出

阶段1的输出是出行点列表：

```
stage1_travel = df[df["is_travel_stage1"]].copy()
stage1_travel[["imsi", "time", "lon", "lat",
"speed"]].to_csv("stage1_travel_points.csv", index=False)
```

4.3 项目实现步骤

1. 计算速度（距离*3600/时间）；
2. 处理边界情况（时间差为0、异常速度）；
3. 判定出行点（速度>8km/h）；
4. 输出阶段1出行点。

4.4 完整代码（速度计算与出行点判定）

```
# ============================================================
# 课时4：速度计算与出行点判定
# ============================================================

import pandas as pd
import numpy as np
import os

# Haversine 函数
def haversine(lon1, lat1, lon2, lat2, unit="km"):
    R = 6371.0
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
    c = 2 * np.arcsin(np.sqrt(a))
    return R * c * (1000 if unit == "m" else 1)


# 读取清洗后数据
RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
cleaned_path = os.path.join(RESULT_DIR, "cleaned_data.csv")
df = pd.read_csv(cleaned_path, dtype={"imsi": str, "time": str,
                                      "lac_id": str, "cell_id": str})
df = df.sort_values(["imsi", "time"]).reset_index(drop=True)
print(f"数据：{len(df)} 行，{df['imsi'].nunique()} 用户")

# ------------------------------------------------------------
# 1. 计算时间差和距离差（复用课时3的逻辑）
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("步骤1：计算时间差和距离差")
print("=" * 60)

df["time_dt"] = pd.to_datetime(df["time"], format="%Y%m%d%H%M%S", errors="coerce")
```

```
df["time_diff"] = df["time_dt"].diff().dt.total_seconds()

df["prev_lon"] = df["lon"].shift(1)
df["prev_lat"] = df["lat"].shift(1)
df["dist_diff"] = haversine(df["prev_lon"], df["prev_lat"], df["lon"], df["lat"])

# 用户边界处理
df["user_change"] = df["imsi"] != df["imsi"].shift(1)
df.loc[df["user_change"], "time_diff"] = np.nan
df.loc[df["user_change"], "dist_diff"] = np.nan

print(f"时间差非空: {df['time_diff'].notna().sum()}")
print(f"距离差非空: {df['dist_diff'].notna().sum()}")

# ------------------------------------------------------------
# 2. 速度计算
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("步骤2: 速度计算")
print("=" * 60)

# 速度 = 距离(km) * 3600 / 时间(秒) = km/h
df["speed"] = df["dist_diff"] * 3600 / df["time_diff"]

print(f"速度统计（原始）:")
print(f"  非空数: {df['speed'].notna().sum()}")
print(f"  最小值: {df['speed'].min():.4f} km/h")
print(f"  最大值: {df['speed'].max():.2f} km/h")
print(f"  中位数: {df['speed'].median():.2f} km/h")
print(f"  平均值: {df['speed'].mean():.2f} km/h")

# ------------------------------------------------------------
# 3. 边界情况处理
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("步骤3: 边界情况处理")
print("=" * 60)
```

```
# 时间差为0 → 速度设为 NaN
zero_time = df["time_diff"] == 0
print(f"时间差为0的点数: {zero_time.sum()}")
df.loc[zero_time, "speed"] = np.nan

# 异常速度（>200km/h）→ 设为 NaN（可能是基站跳跃）
abnormal_speed = df["speed"] > 200
print(f"速度>200km/h的点数: {abnormal_speed.sum()}")
df.loc[abnormal_speed, "speed"] = np.nan

print(f"\n处理后速度统计:")
print(f"  非空数: {df['speed'].notna().sum()}")
print(f"  最小值: {df['speed'].min():.4f} km/h")
print(f"  最大值: {df['speed'].max():.2f} km/h")
print(f"  中位数: {df['speed'].median():.2f} km/h")
print(f"  平均值: {df['speed'].mean():.2f} km/h")

# ------------------------------------------------------------
# 4. 出行点判定
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("步骤4：出行点判定（速度>8km/h）")
print("=" * 60)

TRAVEL_SPEED_THRESHOLD = 8.0   # km/h
df["is_travel_stage1"] = df["speed"] > TRAVEL_SPEED_THRESHOLD

travel_count = df["is_travel_stage1"].sum()
total_valid = df["speed"].notna().sum()
print(f"速度阈值: {TRAVEL_SPEED_THRESHOLD} km/h")
print(f"阶段1出行点数: {travel_count}")
print(f"有效速度点数: {total_valid}")
print(f"出行点占比: {travel_count / total_valid * 100:.1f}%（占有效速度点）")
print(f"出行点占比: {travel_count / len(df) * 100:.1f}%（占总点数）")

# 速度分布
print(f"\n速度分布:")
bins = [0, 2, 4, 6, 8, 12, 20, 50, 100, 200]
```

```
labels = ["0-2", "2-4", "4-6", "6-8", "8-12", "12-20", "20-50", "50-100", "100-200"]
df["speed_bin"] = pd.cut(df["speed"], bins=bins, labels=labels)
speed_dist = df["speed_bin"].value_counts().sort_index()
for label, count in speed_dist.items():
    marker = " ← 出行" if count > 0 and label in ["8-12", "12-20", "20-50", "50-100",
"100-200"] else ""
    print(f"  {label} km/h: {count} ({count/total_valid*100:.1f}%){marker}")

# ------------------------------------------------------------
# 5. 输出阶段1出行点
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("步骤5：输出阶段1出行点")
print("=" * 60)

stage1_travel = df[df["is_travel_stage1"]].copy()
output_cols = ["imsi", "time", "lac_id", "cell_id", "lon", "lat", "speed"]
stage1_travel = stage1_travel[output_cols]

output_path = os.path.join(RESULT_DIR, "stage1_travel_points.csv")
stage1_travel.to_csv(output_path, index=False, encoding="utf-8")

print(f"输出文件：{output_path}")
print(f"出行点数：{len(stage1_travel)}")
print(f"涉及用户数：{stage1_travel['imsi'].nunique()}")
print(f"\n前5行:\n{stage1_travel.head()}")

# ------------------------------------------------------------
# 6. 验证
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("步骤6：验证")
print("=" * 60)

# 所有出行点速度都应>8
print(f"所有出行点速度>8：{(stage1_travel['speed'] > 8).all()}")
print(f"出行点速度范围：[{stage1_travel['speed'].min():.2f},
{stage1_travel['speed'].max():.2f}] km/h")
```

```
print(f"出行点速度中位数: {stage1_travel['speed'].median():.2f} km/h")

# 每个用户的出行点数
user_travel = stage1_travel.groupby("imsi").size().sort_values(ascending=False)
print(f"\n出行点最多的前5个用户:")
print(user_travel.head())

print("\n" + "=" * 60)
print("阶段1速度计算与出行点判定完成！")
print("=" * 60)
```

4.5 运行结果与解读

运行后展示：
- 速度统计（原始和处理后）；
- 边界情况处理（时间差为0、异常速度>200km/h）；
- 出行点判定（速度>8km/h），出行点占比约30%；
- 速度分布（分箱统计）；
- 输出 stage1_travel_points.csv；
- 验证（所有出行点速度>8、用户分布）。

> 关键理解：
> 1. 速度 = 距离 * 3600 / 时间，单位 km/h；
> 2. 时间差为0和异常速度>200km/h设为 NaN；
> 3. 速度>8km/h判出行点，约占30%；
> 4. 输出 stage1_travel_points.csv 供后续使用。

4.6 当堂练习

1. 修改速度阈值为 5 km/h 和 12 km/h，观察出行点数量变化；
2. 统计出行点的时间分布（按小时），分析哪个时段出行最多；
3. 思考：为什么异常速度阈值设为 200 km/h？（提示：中国高铁最高速度约350km/h，但城市内交通一般不超过120km/h，200km/h以上基本是基站跳跃误差）

4.7 课时小结

- 速度 = 距离(km) * 3600 / 时间(秒)，单位 km/h；
- 边界处理：时间差为0、速度>200km/h 设为 NaN；

- 出行点判定：速度>8km/h，约占30%；
- 输出 stage1_travel_points.csv；
- 阶段1出行点不再参与阶段2聚类，直接标记为出行。

课时5：连续段合并与边界处理

5.1 课时目标

深入理解连续段的合并策略、边界点处理、段级统计。

5.2 知识点深度讲解

5.2.1 连续段的合并策略

阶段1识别连续段后，需要对每个连续段做段级分析：
1. **段内速度统计**：段内所有相邻点速度的最大值、平均值、中位数；
2. **段级出行判定**：如果段内有任意一个点速度>8km/h，这个段标记为"出行段"；
3. **段内出行点传播**：出行段内的所有点都标记为出行点（可选，本项目不做）。

为什么需要段级分析?
- 单个点的速度可能因为基站跳跃而异常；
- 段级分析更稳定，可以减少误判；
- 出行段的概念更符合业务逻辑（用户在一段时间内出行）。

5.2.2 边界点处理

连续段的边界点（第一个点和最后一个点）有特殊性：
1. **段首点**：只有后一个相邻点，速度只算一次；
2. **段尾点**：只有前一个相邻点，速度只算一次；
3. **边界点的速度可能不准确**：如果段首点前是低频点，时间间隔可能很大。

处理策略：
- 边界点的速度照常计算，但如果时间间隔过大（>30分钟），速度设为 NaN；
- 边界点不单独判定，跟随段级判定。

5.2.3 段级统计实现

```
# 按段分组统计
segment_stats = df[df["segment_id"] != -1].groupby(["imsi", "segment_id"]).agg(
    points=("lon", "count"),           # 段内点数
    max_speed=("speed", "max"),        # 最大速度
    mean_speed=("speed", "mean"),      # 平均速度
    median_speed=("speed", "median"),  # 中位速度
    start_time=("time", "min"),        # 开始时间
    end_time=("time", "max"),          # 结束时间
    start_lon=("lon", "first"),        # 开始经度
    end_lon=("lon", "last"),           # 结束经度
).reset_index()

# 标记出行段（段内最大速度>8）
segment_stats["is_travel_segment"] = segment_stats["max_speed"] > 8

# 段持续时间（分钟）
segment_stats["duration_min"] = (
    pd.to_datetime(segment_stats["end_time"], format="%Y%m%d%H%M%S") -
    pd.to_datetime(segment_stats["start_time"], format="%Y%m%d%H%M%S")
).dt.total_seconds() / 60
```

5.2.4 出行段 vs 驻留段

特征	出行段	驻留段
段内最大速度	>8 km/h	<=8 km/h
段内点数	通常较少（2-10）	通常较多（10+）
持续时间	通常较短（<30分钟）	通常较长（>30分钟）
空间范围	较大（移动距离远）	较小（聚集在小范围）
业务含义	用户在移动	用户在停留

5.3 项目实现步骤

1. 按段分组统计段级指标；
2. 标记出行段和驻留段；

3. 分析段级特征（点数、持续时间、速度）；

4. 输出段级统计结果。

5.4 完整代码（连续段合并与边界处理）

```
# ============================================================
# 课时5：连续段合并与边界处理
# ============================================================

import pandas as pd
import numpy as np
import os

def haversine(lon1, lat1, lon2, lat2, unit="km"):
    R = 6371.0
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
    c = 2 * np.arcsin(np.sqrt(a))
    return R * c * (1000 if unit == "m" else 1)


RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
cleaned_path = os.path.join(RESULT_DIR, "cleaned_data.csv")
df = pd.read_csv(cleaned_path, dtype={"imsi": str, "time": str,
                                      "lac_id": str, "cell_id": str})
df = df.sort_values(["imsi", "time"]).reset_index(drop=True)

# ------------------------------------------------------------
# 1. 复用前面的逻辑：低频点过滤、连续段识别、速度计算
# ------------------------------------------------------------
print("=" * 60)
print("步骤1：复用前面的逻辑（低频点、连续段、速度）")
print("=" * 60)

# 低频点
df["laci"] = df["lac_id"] + "-" + df["cell_id"]
df["laci_count"] = df.groupby(["imsi", "laci"])["laci"].transform("count")
df["is_low_freq"] = df["laci_count"] < 4
```

```
# 连续段
df["user_change"] = df["imsi"] != df["imsi"].shift(1)
df["prev_low_freq"] = df["is_low_freq"].shift(1).fillna(True)
df["new_segment"] = df["user_change"] | (~df["is_low_freq"] & df["prev_low_freq"])
df["segment_id"] = df.groupby("imsi")["new_segment"].cumsum()
df.loc[df["is_low_freq"], "segment_id"] = -1

# 速度
df["time_dt"] = pd.to_datetime(df["time"], format="%Y%m%d%H%M%S", errors="coerce")
df["time_diff"] = df["time_dt"].diff().dt.total_seconds()
df["prev_lon"] = df["lon"].shift(1)
df["prev_lat"] = df["lat"].shift(1)
df["dist_diff"] = haversine(df["prev_lon"], df["prev_lat"], df["lon"], df["lat"])
df.loc[df["user_change"], "time_diff"] = np.nan
df.loc[df["user_change"], "dist_diff"] = np.nan
df["speed"] = df["dist_diff"] * 3600 / df["time_diff"]
df.loc[df["time_diff"] == 0, "speed"] = np.nan
df.loc[df["speed"] > 200, "speed"] = np.nan

# 连续段边界处速度设为NaN
df["segment_change"] = (df["segment_id"] != df["segment_id"].shift(1)) |
(df["segment_id"] == -1)
df.loc[df["segment_change"], "speed"] = np.nan

print(f"总点数: {len(df)}")
print(f"有效速度点数: {df['speed'].notna().sum()}")
print(f"连续段数: {df[df['segment_id'] != -1].groupby(['imsi',
'segment_id']).ngroups}")

# ------------------------------------------------------------
# 2. 段级统计
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("步骤2: 段级统计")
print("=" * 60)

valid_df = df[df["segment_id"] != -1].copy()
```

```
segment_stats = valid_df.groupby(["imsi", "segment_id"]).agg(
    points=("lon", "count"),
    max_speed=("speed", "max"),
    mean_speed=("speed", "mean"),
    median_speed=("speed", "median"),
    start_time=("time", "min"),
    end_time=("time", "max"),
    start_lon=("lon", "first"),
    start_lat=("lat", "first"),
    end_lon=("lon", "last"),
    end_lat=("lat", "last"),
).reset_index()

# 段持续时间（分钟）
segment_stats["duration_min"] = (
    pd.to_datetime(segment_stats["end_time"], format="%Y%m%d%H%M%S") -
    pd.to_datetime(segment_stats["start_time"], format="%Y%m%d%H%M%S")
).dt.total_seconds() / 60

# 段移动距离（起点到终点的直线距离）
segment_stats["segment_dist"] = haversine(
    segment_stats["start_lon"], segment_stats["start_lat"],
    segment_stats["end_lon"], segment_stats["end_lat"]
)

# 标记出行段（段内最大速度>8）
segment_stats["is_travel_segment"] = segment_stats["max_speed"] > 8

print(f"段级统计:")
print(f"  总段数: {len(segment_stats)}")
print(f"  出行段数: {segment_stats['is_travel_segment'].sum()}")
print(f"  驻留段数: {(~segment_stats['is_travel_segment']).sum()}")

# ------------------------------------------------------------
# 3. 出行段 vs 驻留段特征对比
# ------------------------------------------------------------
print("\n" + "=" * 60)
```

```
print("步骤3：出行段 vs 驻留段特征对比")
print("=" * 60)

travel_segs = segment_stats[segment_stats["is_travel_segment"]]
stay_segs = segment_stats[~segment_stats["is_travel_segment"]]

print(f"{'特征':<15} {'出行段':<15} {'驻留段':<15}")
print("-" * 45)
print(f"{'段数':<15} {len(travel_segs):<15} {len(stay_segs):<15}")
print(f"{'平均点数':<15} {travel_segs['points'].mean():<15.1f}
{stay_segs['points'].mean():<15.1f}")
print(f"{'平均持续(分)':<15} {travel_segs['duration_min'].mean():<15.1f}
{stay_segs['duration_min'].mean():<15.1f}")
print(f"{'平均最大速度':<15} {travel_segs['max_speed'].mean():<15.2f}
{stay_segs['max_speed'].mean():<15.2f}")
print(f"{'平均移动距离':<15} {travel_segs['segment_dist'].mean():<15.2f}
{stay_segs['segment_dist'].mean():<15.2f}")

# ------------------------------------------------------------
# 4. 边界点分析
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("步骤4：边界点分析")
print("=" * 60)

# 段首点和段尾点
boundary_points = valid_df.groupby(["imsi", "segment_id"]).agg(
    first_speed=("speed", "first"),
    last_speed=("speed", "last"),
    first_time_diff=("time_diff", "first"),
    last_time_diff=("time_diff", "last"),
).reset_index()

print(f"段首点速度统计:")
print(f"  非空数: {boundary_points['first_speed'].notna().sum()}")
print(f"  中位数: {boundary_points['first_speed'].median():.2f} km/h")
print(f"  平均值: {boundary_points['first_speed'].mean():.2f} km/h")
```

```
print(f"\n段尾点速度统计:")
print(f"  非空数: {boundary_points['last_speed'].notna().sum()}")
print(f"  中位数: {boundary_points['last_speed'].median():.2f} km/h")
print(f"  平均值: {boundary_points['last_speed'].mean():.2f} km/h")

# ------------------------------------------------------------
# 5. 输出段级统计
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("步骤5: 输出段级统计")
print("=" * 60)

output_cols = ["imsi", "segment_id", "points", "max_speed", "mean_speed",
               "duration_min", "segment_dist", "is_travel_segment",
               "start_time", "end_time"]
segment_output = segment_stats[output_cols]
output_path = os.path.join(RESULT_DIR, "stage1_segments.csv")
segment_output.to_csv(output_path, index=False, encoding="utf-8")
print(f"输出文件: {output_path}")
print(f"段数: {len(segment_output)}")

print("\n" + "=" * 60)
print("连续段合并与边界处理完成！")
print("=" * 60)
```

5.5 运行结果与解读

运行后展示：
- 段级统计（总段数、出行段数、驻留段数）；
- 出行段 vs 驻留段特征对比（点数、持续时间、速度、移动距离）；
- 边界点分析（段首/段尾速度统计）；
- 输出 stage1_segments.csv。

> 关键理解：
> 1. 出行段通常点数少、持续时间短、移动距离远；
> 2. 驻留段通常点数多、持续时间长、移动距离近；
> 3. 段级分析比单点分析更稳定，可以减少误判。

5.6 当堂练习

1. 统计出行段和驻留段的持续时间分布，画直方图；
2. 思考：为什么出行段的平均点数比驻留段少？（提示：出行时用户移动快，基站切换频繁，每个基站停留时间短）
3. 尝试"出行段内所有点都标记为出行点"的策略，对比与单点判定的结果差异。

5.7 课时小结

- 段级统计：按段分组计算点数、速度、持续时间、移动距离；
- 出行段判定：段内最大速度>8km/h；
- 出行段 vs 驻留段：出行段点数少、持续短、移动远；驻留段点数多、持续长、移动近；
- 边界点：段首/段尾只有一个相邻点，速度可能不准确；
- 输出 stage1_segments.csv 供分析使用。

课时6：阶段1完整实现（tr_stage1.py）

6.1 课时目标

将前面课时的所有逻辑整合为一个完整的、可运行的阶段1模块 tr_stage1.py。

6.2 知识点深度讲解

6.2.1 模块结构设计

tr_stage1.py 的结构：
1. **配置区**：路径、阈值参数；
2. **工具函数**：Haversine；
3. **主函数 stage1()**：按步骤执行所有逻辑；
4. **if name == "main"**：入口。

6.2.2 完整流程

```
读取清洗后数据
  ↓
1. 低频点过滤（laci出现次数<4）
  ↓
2. 连续段识别（cumsum实现段ID）
  ↓
3. 时间差计算（datetime diff）
  ↓
4. 距离差计算（shift + Haversine）
  ↓
5. 用户边界和连续段边界处理（差值设为NaN）
  ↓
6. 速度计算（距离*3600/时间）
  ↓
7. 边界情况处理（时间差为0、速度>200）
  ↓
8. 出行点判定（速度>8km/h）
  ↓
9. 段级统计
  ↓
输出 stage1_travel_points.csv + stage1_segments.csv
```

6.3 项目实现步骤

按上述流程编写完整代码，每步打印进度和统计。

6.4 完整代码（tr_stage1.py）

```
# ============================================================
# 阶段1模块：tr_stage1.py
# 功能：低频点过滤、连续段识别、速度计算、出行点判定
# 运行：python tr_stage1.py
# 输入：cleaned_data.csv
# 输出：stage1_travel_points.csv, stage1_segments.csv
# ============================================================

import os
import pandas as pd
import numpy as np

# ------------------------------------------------------------
# 配置
# ------------------------------------------------------------
RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
os.makedirs(RESULT_DIR, exist_ok=True)

# 阈值参数
LOW_FREQ_THRESHOLD = 4          # 低频点阈值（出现次数）
TRAVEL_SPEED_THRESHOLD = 8.0    # 出行速度阈值（km/h）
ABNORMAL_SPEED_THRESHOLD = 200  # 异常速度阈值（km/h）


def haversine(lon1, lat1, lon2, lat2, unit="km"):
    """计算球面距离（向量化）。"""
    R = 6371.0
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
    c = 2 * np.arcsin(np.sqrt(a))
    d = R * c
    return d * 1000 if unit == "m" else d
```

```
def stage1():
    """执行阶段1完整流程。"""

    # ============================================================
    # 步骤0：读取数据
    # ============================================================
    print("=" * 60)
    print("步骤0：读取清洗后数据")
    print("=" * 60)

    cleaned_path = os.path.join(RESULT_DIR, "cleaned_data.csv")
    if not os.path.exists(cleaned_path):
        print(f"错误：{cleaned_path} 不存在，请先运行 data_clean.py")
        return None

    df = pd.read_csv(cleaned_path, dtype={"imsi": str, "time": str,
                                          "lac_id": str, "cell_id": str})
    df = df.sort_values(["imsi", "time"]).reset_index(drop=True)
    print(f"  数据：{len(df)} 行，{df['imsi'].nunique()} 用户")

    # ============================================================
    # 步骤1：低频点过滤
    # ============================================================
    print("\n" + "=" * 60)
    print(f"步骤1：低频点过滤（出现次数<{LOW_FREQ_THRESHOLD}）")
    print("=" * 60)

    df["laci"] = df["lac_id"] + "-" + df["cell_id"]
    df["laci_count"] = df.groupby(["imsi", "laci"])["laci"].transform("count")
    df["is_low_freq"] = df["laci_count"] < LOW_FREQ_THRESHOLD
    print(f"  低频点数：{df['is_low_freq'].sum()}
({df['is_low_freq'].mean()*100:.1f}%)")

    # ============================================================
    # 步骤2：连续段识别
    # ============================================================
    print("\n" + "=" * 60)
```

```
print("步骤2：连续段识别")
print("=" * 60)

df["user_change"] = df["imsi"] != df["imsi"].shift(1)
df["prev_low_freq"] = df["is_low_freq"].shift(1).fillna(True)
df["new_segment"] = df["user_change"] | (~df["is_low_freq"] & df["prev_low_freq"])
df["segment_id"] = df.groupby("imsi")["new_segment"].cumsum()
df.loc[df["is_low_freq"], "segment_id"] = -1

segment_count = df[df["segment_id"] != -1].groupby(["imsi", "segment_id"]).ngroups
print(f"  连续段数：{segment_count}")

# ============================================================
# 步骤3：时间差计算
# ============================================================
print("\n" + "=" * 60)
print("步骤3：时间差计算")
print("=" * 60)

df["time_dt"] = pd.to_datetime(df["time"], format="%Y%m%d%H%M%S", errors="coerce")
df["time_diff"] = df["time_dt"].diff().dt.total_seconds()
print(f"  时间差中位数：{df['time_diff'].median():.0f} 秒")

# ============================================================
# 步骤4：距离差计算
# ============================================================
print("\n" + "=" * 60)
print("步骤4：距离差计算（向量化 Haversine）")
print("=" * 60)

df["prev_lon"] = df["lon"].shift(1)
df["prev_lat"] = df["lat"].shift(1)
df["dist_diff"] = haversine(df["prev_lon"], df["prev_lat"], df["lon"], df["lat"])
print(f"  距离差中位数：{df['dist_diff'].median():.4f} km")

# ============================================================
# 步骤5：边界处理
# ============================================================
```

```
    print("\n" + "=" * 60)
    print("步骤5：用户边界和连续段边界处理")
    print("=" * 60)

    # 用户边界
    df.loc[df["user_change"], "time_diff"] = np.nan
    df.loc[df["user_change"], "dist_diff"] = np.nan

    # 连续段边界
    df["segment_change"] = (df["segment_id"] != df["segment_id"].shift(1)) |
(df["segment_id"] == -1)
    df.loc[df["segment_change"], "time_diff"] = np.nan
    df.loc[df["segment_change"], "dist_diff"] = np.nan

    print(f"  有效时间差数：{df['time_diff'].notna().sum()}")
    print(f"  有效距离差数：{df['dist_diff'].notna().sum()}")

    # ============================================================
    # 步骤6：速度计算
    # ============================================================
    print("\n" + "=" * 60)
    print("步骤6：速度计算")
    print("=" * 60)

    df["speed"] = df["dist_diff"] * 3600 / df["time_diff"]
    print(f"  速度中位数：{df['speed'].median():.2f} km/h")
    print(f"  速度平均值：{df['speed'].mean():.2f} km/h")

    # ============================================================
    # 步骤7：边界情况处理
    # ============================================================
    print("\n" + "=" * 60)
    print("步骤7：边界情况处理")
    print("=" * 60)

    zero_time = df["time_diff"] == 0
    abnormal = df["speed"] > ABNORMAL_SPEED_THRESHOLD
    print(f"  时间差为0：{zero_time.sum()} 点")
```

```
    print(f"  速度>{ABNORMAL_SPEED_THRESHOLD}km/h: {abnormal.sum()} 点")

    df.loc[zero_time, "speed"] = np.nan
    df.loc[abnormal, "speed"] = np.nan
    print(f"  处理后有效速度数: {df['speed'].notna().sum()}")

    # ============================================================
    # 步骤8: 出行点判定
    # ============================================================
    print("\n" + "=" * 60)
    print(f"步骤8: 出行点判定（速度>{TRAVEL_SPEED_THRESHOLD}km/h）")
    print("=" * 60)

    df["is_travel_stage1"] = df["speed"] > TRAVEL_SPEED_THRESHOLD
    travel_count = df["is_travel_stage1"].sum()
    print(f"  阶段1出行点数: {travel_count}")
    print(f"  占总点数: {travel_count / len(df) * 100:.1f}%")
    print(f"  涉及用户数: {df[df['is_travel_stage1']]['imsi'].nunique()}")

    # ============================================================
    # 步骤9: 段级统计
    # ============================================================
    print("\n" + "=" * 60)
    print("步骤9: 段级统计")
    print("=" * 60)

    valid_df = df[df["segment_id"] != -1]
    segment_stats = valid_df.groupby(["imsi", "segment_id"]).agg(
        points=("lon", "count"),
        max_speed=("speed", "max"),
        mean_speed=("speed", "mean"),
        start_time=("time", "min"),
        end_time=("time", "max"),
    ).reset_index()
    segment_stats["is_travel_segment"] = segment_stats["max_speed"] >
TRAVEL_SPEED_THRESHOLD
    print(f"  总段数: {len(segment_stats)}")
    print(f"  出行段: {segment_stats['is_travel_segment'].sum()}")
```

```
    print(f"  驻留段: {(~segment_stats['is_travel_segment']).sum()}")

    # ============================================================
    # 输出
    # ============================================================
    print("\n" + "=" * 60)
    print("输出结果")
    print("=" * 60)

    # 出行点
    travel_output = df[df["is_travel_stage1"]][
        ["imsi", "time", "lac_id", "cell_id", "lon", "lat", "speed"]
    ].copy()
    travel_path = os.path.join(RESULT_DIR, "stage1_travel_points.csv")
    travel_output.to_csv(travel_path, index=False, encoding="utf-8")
    print(f"  出行点: {travel_path} ({len(travel_output)} 行)")

    # 段级统计
    segment_path = os.path.join(RESULT_DIR, "stage1_segments.csv")
    segment_stats.to_csv(segment_path, index=False, encoding="utf-8")
    print(f"  段级统计: {segment_path} ({len(segment_stats)} 行)")

    # 完整数据（含阶段1标签，供阶段2使用）
    full_path = os.path.join(RESULT_DIR, "stage1_full.csv")
    df[["imsi", "time", "lac_id", "cell_id", "lon", "lat",
        "laci", "laci_count", "is_low_freq", "segment_id",
        "time_diff", "dist_diff", "speed", "is_travel_stage1"]
    ].to_csv(full_path, index=False, encoding="utf-8")
    print(f"  完整数据: {full_path} ({len(df)} 行)")

    print("\n" + "=" * 60)
    print("阶段1完成！")
    print("=" * 60)

    return df
```

```
if __name__ == "__main__":
    stage1()
```

6.5 运行结果与解读

运行 `D:\soft\python\python.exe tr_stage1.py` 后应看到：
- 每步的进度和统计；
- 最终出行点数约3000-4000（抽样2万行清洗后）；
- 输出三个文件：stage1_travel_points.csv、stage1_segments.csv、stage1_full.csv。

> 关键验证：
> 1. 出行点速度都>8km/h；
> 2. 出行点涉及多个用户；
> 3. stage1_full.csv 包含所有中间字段，供阶段2使用。

6.6 当堂练习

1. 运行 tr_stage1.py，记录每步的输出；
2. 检查 stage1_travel_points.csv 的前10行，验证速度都>8；
3. 思考：为什么要输出 stage1_full.csv（包含所有中间字段）？（提示：阶段2需要 segment_id、is_low_freq 等字段）

6.7 课时小结

- tr_stage1.py 整合了所有阶段1逻辑；
- 9个步骤按顺序执行，每步打印进度；
- 输出三个文件：出行点、段级统计、完整数据；
- 完整数据供阶段2使用。

课时7：阶段1结果分析与可视化

7.1 课时目标

对阶段1结果进行深入分析，并用 matplotlib 可视化（速度分布、出行点空间分布、时间分布）。

7.2 知识点深度讲解

7.2.1 分析维度

1. **速度分布**：所有点的速度分布、出行点的速度分布；
2. **空间分布**：出行点的经纬度分布（散点图）；
3. **时间分布**：出行点按小时的分布；
4. **用户分布**：每个用户的出行点数分布；
5. **连续段分布**：段长度、持续时间的分布。

7.2.2 matplotlib 基础

```
import matplotlib.pyplot as plt

# 设置中文字体
plt.rcParams["font.sans-serif"] = ["SimHei"]   # Windows
plt.rcParams["axes.unicode_minus"] = False      # 负号显示

# 直方图
plt.hist(data, bins=50, edgecolor="black")
plt.xlabel("速度 (km/h)")
plt.ylabel("频数")
plt.title("速度分布")
plt.savefig("speed_hist.png", dpi=150, bbox_inches="tight")
plt.close()

# 散点图
plt.scatter(lon, lat, c=speed, cmap="viridis", s=1)
plt.colorbar(label="速度 (km/h)")
plt.xlabel("经度")
plt.ylabel("纬度")
plt.title("出行点空间分布")
plt.savefig("travel_scatter.png", dpi=150, bbox_inches="tight")
plt.close()

# 柱状图
plt.bar(hours, counts)
plt.xlabel("小时")
plt.ylabel("出行点数")
plt.title("出行点时间分布")
plt.savefig("travel_hour_bar.png", dpi=150, bbox_inches="tight")
plt.close()
```

7.3 项目实现步骤

1. 读取阶段1结果；
2. 绘制速度分布直方图；
3. 绘制出行点空间分布散点图；

4. 绘制出行点时间分布柱状图；

5. 绘制用户出行点数分布；

6. 保存所有图片。

7.4 完整代码（阶段1结果分析与可视化）

```
# ============================================================
# 课时7：阶段1结果分析与可视化
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

# 设置中文字体
plt.rcParams["font.sans-serif"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
FIG_DIR = os.path.join(RESULT_DIR, "figures")
os.makedirs(FIG_DIR, exist_ok=True)

# 读取阶段1完整数据
full_path = os.path.join(RESULT_DIR, "stage1_full.csv")
if not os.path.exists(full_path):
    print("请先运行 tr_stage1.py")
    exit()

df = pd.read_csv(full_path, dtype={"imsi": str, "time": str,
                                   "lac_id": str, "cell_id": str, "laci": str})
print(f"数据：{len(df)} 行，{df['imsi'].nunique()} 用户")

travel_df = df[df["is_travel_stage1"] == True].copy()
print(f"阶段1出行点：{len(travel_df)} 行")

# ------------------------------------------------------------
# 图1：速度分布直方图
# ------------------------------------------------------------
print("\n生成图1：速度分布直方图")
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

```
# 所有有效速度
valid_speeds = df[df["speed"].notna()]["speed"]
axes[0].hist(valid_speeds, bins=50, edgecolor="black", alpha=0.7)
axes[0].axvline(x=8, color="red", linestyle="--", label="出行阈值=8km/h")
axes[0].set_xlabel("速度 (km/h)")
axes[0].set_ylabel("频数")
axes[0].set_title(f"所有点速度分布 (n={len(valid_speeds)})")
axes[0].legend()
axes[0].set_xlim(0, 100)

# 出行点速度
axes[1].hist(travel_df["speed"], bins=30, edgecolor="black", alpha=0.7, color="orange")
axes[1].set_xlabel("速度 (km/h)")
axes[1].set_ylabel("频数")
axes[1].set_title(f"出行点速度分布 (n={len(travel_df)})")
axes[1].set_xlim(0, 100)

plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_stage1_speed_hist.png"), dpi=150,
bbox_inches="tight")
plt.close()
print("  保存: fig_stage1_speed_hist.png")

# ------------------------------------------------------------
# 图2: 出行点空间分布散点图
# ------------------------------------------------------------
print("生成图2: 出行点空间分布散点图")
fig, ax = plt.subplots(figsize=(10, 8))

# 所有点（灰色背景）
ax.scatter(df["lon"], df["lat"], c="lightgray", s=1, alpha=0.5, label="所有点")

# 出行点（按速度着色）
scatter = ax.scatter(travel_df["lon"], travel_df["lat"],
                     c=travel_df["speed"], cmap="YlOrRd", s=5, alpha=0.8, label="出行
点")
plt.colorbar(scatter, label="速度 (km/h)")
```

```
ax.set_xlabel("经度")
ax.set_ylabel("纬度")
ax.set_title(f"阶段1出行点空间分布 (n={len(travel_df)})")
ax.legend()

plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_stage1_travel_scatter.png"), dpi=150,
bbox_inches="tight")
plt.close()
print("  保存: fig_stage1_travel_scatter.png")

# ------------------------------------------------------------
# 图3：出行点时间分布柱状图
# ------------------------------------------------------------
print("生成图3：出行点时间分布柱状图")
travel_df["hour"] = travel_df["time"].str[8:10].astype(int)
hour_dist = travel_df.groupby("hour").size().reindex(range(24), fill_value=0)

fig, ax = plt.subplots(figsize=(12, 5))
ax.bar(hour_dist.index, hour_dist.values, color="steelblue", edgecolor="black")
ax.set_xlabel("小时")
ax.set_ylabel("出行点数")
ax.set_title("阶段1出行点时间分布（按小时）")
ax.set_xticks(range(24))
ax.grid(axis="y", alpha=0.3)

plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_stage1_travel_hour.png"), dpi=150,
bbox_inches="tight")
plt.close()
print("  保存: fig_stage1_travel_hour.png")

# ------------------------------------------------------------
# 图4：用户出行点数分布
# ------------------------------------------------------------
print("生成图4：用户出行点数分布")
user_travel = travel_df.groupby("imsi").size().sort_values(ascending=False)
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Top20用户
axes[0].bar(range(20), user_travel.head(20).values, color="coral", edgecolor="black")
axes[0].set_xlabel("用户排名")
axes[0].set_ylabel("出行点数")
axes[0].set_title(f"出行点数 Top20 用户（共{user_travel.shape[0]}用户）")
axes[0].set_xticks(range(0, 20, 2))

# 分布直方图
axes[1].hist(user_travel.values, bins=20, edgecolor="black", alpha=0.7, color="green")
axes[1].set_xlabel("用户出行点数")
axes[1].set_ylabel("用户数")
axes[1].set_title("用户出行点数分布")

plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_stage1_user_travel.png"), dpi=150,
bbox_inches="tight")
plt.close()
print("  保存: fig_stage1_user_travel.png")

# ------------------------------------------------------------
# 统计摘要
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("阶段1结果统计摘要")
print("=" * 60)
print(f"总点数: {len(df)}")
print(f"有效速度点数: {df['speed'].notna().sum()}")
print(f"阶段1出行点数: {len(travel_df)} ({len(travel_df)/len(df)*100:.1f}%)")
print(f"出行点速度: 中位数={travel_df['speed'].median():.2f}, 平均=
{travel_df['speed'].mean():.2f} km/h")
print(f"出行高峰时段: {hour_dist.idxmax()}时 ({hour_dist.max()}点)")
print(f"涉及用户数: {travel_df['imsi'].nunique()}")
print(f"人均出行点数: {len(travel_df)/travel_df['imsi'].nunique():.1f}")
print(f"\n图片保存在: {FIG_DIR}")
```

7.5 运行结果与解读

运行后生成4张图：
1. 速度分布直方图（所有点 vs 出行点）；
2. 出行点空间分布散点图（按速度着色）；
3. 出行点时间分布柱状图（按小时）；
4. 用户出行点数分布（Top20 + 直方图）。

> 关键发现：
> 1. 速度分布呈右偏，大部分点速度<8km/h（驻留），出行点速度集中在8-50km/h；
> 2. 出行点空间分布沿主要交通干道（地铁、公交）；
> 3. 出行高峰时段通常在早8-9点和晚17-18点；
> 4. 用户出行点数分布不均，少数用户出行点多。

7.6 当堂练习

1. 运行可视化代码，查看生成的4张图；
2. 在散点图上叠加商业区位置，分析出行点是否集中在商圈附近；
3. 思考：为什么出行点速度集中在8-50km/h，而不是更高？（提示：城市内交通速度有限，地铁约30-40km/h，公交约20-30km/h）

7.7 课时小结

- 速度分布：右偏，大部分<8km/h，出行点8-50km/h；
- 空间分布：出行点沿交通干道；
- 时间分布：早晚高峰；
- 用户分布：不均，少数用户出行点多；
- 4张图保存到 figures 目录。

课时8：参数调优与边界情况

8.1 课时目标

理解阶段1各参数的影响，掌握参数调优方法，处理边界情况。

8.2 知识点深度讲解

8.2.1 参数敏感性分析

阶段1有3个关键参数：
1. **低频点阈值**（默认4）：影响连续段数量和速度计算范围；
2. **出行速度阈值**（默认8km/h）：直接影响出行点数量；
3. **异常速度阈值**（默认200km/h）：影响异常值过滤。

调优方法：
- 固定其他参数，变化一个参数，观察结果变化；
- 画出参数-结果曲线，找到拐点；
- 结合业务知识选择合理值。

8.2.2 出行速度阈值的影响

速度阈值是最敏感的参数：
- 阈值太低（如4km/h）：步行也被判为出行，出行点过多；
- 阈值太高（如15km/h）：自行车、慢速公交被判为驻留，出行点过少；
- 8km/h 是合理的平衡点（步行约4-5km/h，慢跑约6-8km/h）。

8.2.3 低频点阈值的影响

- 阈值太低（如2）：低频点少，连续段多，但噪声大；
- 阈值太高（如8）：低频点多，连续段少，可能误删真实驻留点；
- 4 是合理的平衡点。

8.2.4 边界情况处理

1. **新用户**：第一个点没有前一个点，时间差和距离差为 NaN；
2. **单段单用户**：用户只有一个连续段，段内点数可能很少；
3. **全低频用户**：用户所有点都是低频点，没有连续段，不参与速度计算；
4. **全出行用户**：用户所有点速度都>8，全部标记为出行；
5. **全驻留用户**：用户所有点速度都<=8，全部标记为驻留。

8.3 项目实现步骤

1. 编写参数敏感性分析代码；
2. 变化速度阈值，观察出行点数量变化；
3. 变化低频点阈值，观察连续段数量变化；
4. 处理边界情况。

8.4 完整代码（参数调优与边界情况）

```
# ============================================================
# 课时8：参数调优与边界情况
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

plt.rcParams["font.sans-serif"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
FIG_DIR = os.path.join(RESULT_DIR, "figures")

def haversine(lon1, lat1, lon2, lat2, unit="km"):
    R = 6371.0
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
    c = 2 * np.arcsin(np.sqrt(a))
    return R * c * (1000 if unit == "m" else 1)


# 读取清洗后数据
cleaned_path = os.path.join(RESULT_DIR, "cleaned_data.csv")
df = pd.read_csv(cleaned_path, dtype={"imsi": str, "time": str,
                                      "lac_id": str, "cell_id": str})
df = df.sort_values(["imsi", "time"]).reset_index(drop=True)
print(f"数据：{len(df)} 行，{df['imsi'].nunique()} 用户")

# 预计算速度（用默认参数）
df["laci"] = df["lac_id"] + "-" + df["cell_id"]
df["laci_count"] = df.groupby(["imsi", "laci"])["laci"].transform("count")
```

```
df["is_low_freq"] = df["laci_count"] < 4
df["user_change"] = df["imsi"] != df["imsi"].shift(1)
df["prev_low_freq"] = df["is_low_freq"].shift(1).fillna(True)
df["new_segment"] = df["user_change"] | (~df["is_low_freq"] & df["prev_low_freq"])
df["segment_id"] = df.groupby("imsi")["new_segment"].cumsum()
df.loc[df["is_low_freq"], "segment_id"] = -1
df["time_dt"] = pd.to_datetime(df["time"], format="%Y%m%d%H%M%S", errors="coerce")
df["time_diff"] = df["time_dt"].diff().dt.total_seconds()
df["prev_lon"] = df["lon"].shift(1)
df["prev_lat"] = df["lat"].shift(1)
df["dist_diff"] = haversine(df["prev_lon"], df["prev_lat"], df["lon"], df["lat"])
df.loc[df["user_change"], "time_diff"] = np.nan
df.loc[df["user_change"], "dist_diff"] = np.nan
df["segment_change"] = (df["segment_id"] != df["segment_id"].shift(1)) |
(df["segment_id"] == -1)
df.loc[df["segment_change"], "time_diff"] = np.nan
df.loc[df["segment_change"], "dist_diff"] = np.nan
df["speed"] = df["dist_diff"] * 3600 / df["time_diff"]
df.loc[df["time_diff"] == 0, "speed"] = np.nan
df.loc[df["speed"] > 200, "speed"] = np.nan

# ------------------------------------------------------------
# 1. 速度阈值敏感性分析
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("1. 速度阈值敏感性分析")
print("=" * 60)

thresholds = [2, 4, 6, 8, 10, 12, 15, 20, 25, 30]
travel_counts = []
for t in thresholds:
    count = (df["speed"] > t).sum()
    travel_counts.append(count)
    print(f"  阈值={t:>2} km/h: 出行点={count:>5} ({count/len(df)*100:>5.1f}%)")

# 画图
fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(thresholds, travel_counts, "o-", color="steelblue", linewidth=2, markersize=8)
```

```
ax.axvline(x=8, color="red", linestyle="--", label="默认阈值=8")
ax.set_xlabel("速度阈值 (km/h)")
ax.set_ylabel("出行点数")
ax.set_title("速度阈值敏感性分析")
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_stage1_speed_sensitivity.png"), dpi=150,
bbox_inches="tight")
plt.close()
print("  保存: fig_stage1_speed_sensitivity.png")

# ------------------------------------------------------------
# 2. 低频点阈值敏感性分析
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("2. 低频点阈值敏感性分析")
print("=" * 60)

low_freq_thresholds = [2, 3, 4, 5, 6, 8, 10]
for t in low_freq_thresholds:
    is_low = df["laci_count"] < t
    segment_count = df[~is_low].groupby(["imsi", "laci"]).ngroups  # 近似
    print(f"  阈值={t:>2}: 低频点={is_low.sum():>5} ({is_low.mean()*100:>5.1f}%)")

# ------------------------------------------------------------
# 3. 边界情况分析
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("3. 边界情况分析")
print("=" * 60)

# 全低频用户
user_low_freq = df.groupby("imsi")["is_low_freq"].all()
all_low_users = user_low_freq[user_low_freq].index
print(f"全低频用户数: {len(all_low_users)}（所有点都是低频点，不参与速度计算）")

# 全出行用户
```

```
user_travel = df[df["speed"].notna()].groupby("imsi").apply(
    lambda x: (x["speed"] > 8).all()
)
all_travel_users = user_travel[user_travel].index if len(user_travel) > 0 else []
print(f"全出行用户数: {len(all_travel_users)}（所有有效点速度都>8）")

# 全驻留用户
user_stay = df[df["speed"].notna()].groupby("imsi").apply(
    lambda x: (x["speed"] <= 8).all()
)
all_stay_users = user_stay[user_stay].index if len(user_stay) > 0 else []
print(f"全驻留用户数: {len(all_stay_users)}（所有有效点速度都<=8）")

# 单段用户
user_segments = df[df["segment_id"] != -1].groupby("imsi")["segment_id"].nunique()
single_segment_users = user_segments[user_segments == 1].index
print(f"单段用户数: {len(single_segment_users)}（只有一个连续段）")

# ------------------------------------------------------------
# 4. 参数推荐
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("4. 参数推荐")
print("=" * 60)
print("  低频点阈值: 4（出现次数<4为低频点）")
print("  出行速度阈值: 8 km/h（步行约4-5，慢跑约6-8，8以上基本是交通工具）")
print("  异常速度阈值: 200 km/h（城市内交通一般不超过120，200以上基本是基站跳跃）")
print("\n  调优建议：")
print("  - 如果出行点过多，提高速度阈值到10-12")
print("  - 如果出行点过少，降低速度阈值到6")
print("  - 如果噪声大，提高低频点阈值到6")
print("  - 如果误删驻留点，降低低频点阈值到2-3")

print("\n" + "=" * 60)
print("阶段1参数调优与边界情况分析完成！")
print("=" * 60)
```

8.5 运行结果与解读

运行后展示：
- 速度阈值敏感性分析（阈值从2到30，出行点数量递减）；
- 低频点阈值敏感性分析；
- 边界情况分析（全低频、全出行、全驻留、单段用户）；
- 参数推荐和调优建议。

> 关键理解：
> 1. 速度阈值是最敏感的参数，8km/h是合理平衡点；
> 2. 低频点阈值影响连续段数量，4是合理平衡点；
> 3. 边界情况（全低频、全出行等）需要特殊处理，但本项目简化处理。

8.6 当堂练习

1. 把速度阈值设为5和12，分别运行 tr_stage1.py，对比出行点数量和可视化结果；
2. 思考：为什么速度阈值敏感性曲线在8-12之间有拐点？（提示：这个区间是步行/慢跑与交通工具的分界）
3. 尝试实现"出行段内所有点都标记为出行点"的策略，对比与单点判定的结果差异。

8.7 课时小结

- 速度阈值最敏感，8km/h是合理平衡点；
- 低频点阈值影响连续段数量，4是合理平衡点；
- 边界情况：全低频、全出行、全驻留、单段用户；
- 调优建议：出行点过多提高阈值，过少降低阈值；
- 阶段1完成，进入阶段2 k-means聚类。

三、本日小结

1. **三阶段算法总览**：速度粗筛→k-means聚类→规则精判，由粗到精；
2. **低频点过滤**：出现次数<4为低频点，不参与速度计算但进入阶段2；
3. **连续段识别**：用 cumsum 实现段ID，用户变化或低频点打断；
4. **时间差与距离差**：datetime diff + shift + Haversine，向量化计算；
5. **速度计算与出行点判定**：速度=距离*3600/时间，>8km/h判出行；
6. **连续段合并与边界处理**：段级统计，出行段vs驻留段特征对比；

7. **完整实现 tr_stage1.py**：9步流程，输出3个文件；
8. **结果分析与可视化**：速度分布、空间分布、时间分布、用户分布；
9. **参数调优**：速度阈值最敏感，8km/h是合理平衡点。

四、本日作业

1. 运行 tr_stage1.py，记录每步输出和最终出行点数；
2. 运行可视化代码，查看4张图，写一段分析（速度分布特征、出行高峰时段、空间分布规律）；
3. 把速度阈值改为6和12，分别运行，对比出行点数量变化，分析哪个阈值更合理；
4. 预习 Day05：思考"k-means 聚类的原理是什么？为什么需要用邓恩指数选择最优K？"